

Vi bytter ut Terraform

Her er hvorfor, hva vi gjør i stedet, og hva vi har lært så langt

Hvem her bruker Terraform i dag?

Hvor mange så tittelen på sesjonen og tenkte “dette høres ut som en fryktelig dårlig ide” ?

> whoami



- Azure MVP, Infrastructure as Code
- Manager, Sopra Steria
- Tech Lead Azure, Statnett
- https://trond.sjovang.no
- https://github.com/sjovang

Agenda

- Hva holder vi egentlig på med?
- Er dette et teknologiproblem da?
- Crossplane for n00bs
- Det viktigste vi har lært

> history

- Azure Landing Zones med Terraform
  - Den gamle modulen (den som inneholdt ca alt i hele verden)
  - Ikke Azure Verified Modules
    - Tidkrevende og tungt å oppdatere til AVM
  - Hva gjør egentlig alle disse Azure Policyene?

- GitLab + CI/CD
  - Noen av repo-ene har vokst seg store, apply tar laaaaaang tid
  - For mange sterke koblinger, vanskelig å løsrive enkeltdeler
  - Pipelines som kjøres sjeldent == potensiale for store avvik
  - Brukere må vente på godkjenning fra plattform-teamet for enkelte endringer

> if [! $technologyproblem]; then 🤷🏼; done

- Kunne vi fikset opp i dette uten store teknologivalg?
  - Løse opp i sterke koblinger
  - Kvitte oss med utdaterte terraform-moduler
  - Forbedre CI/CD-løsningene
- Problemet er egentlig … Brukerne
  - De liker jo faktisk ikke terraform noe særlig
  - De liker i alle fall ikke å vente på CI/CD
  - Eller på at plattform-teamet skal godkjenne noe

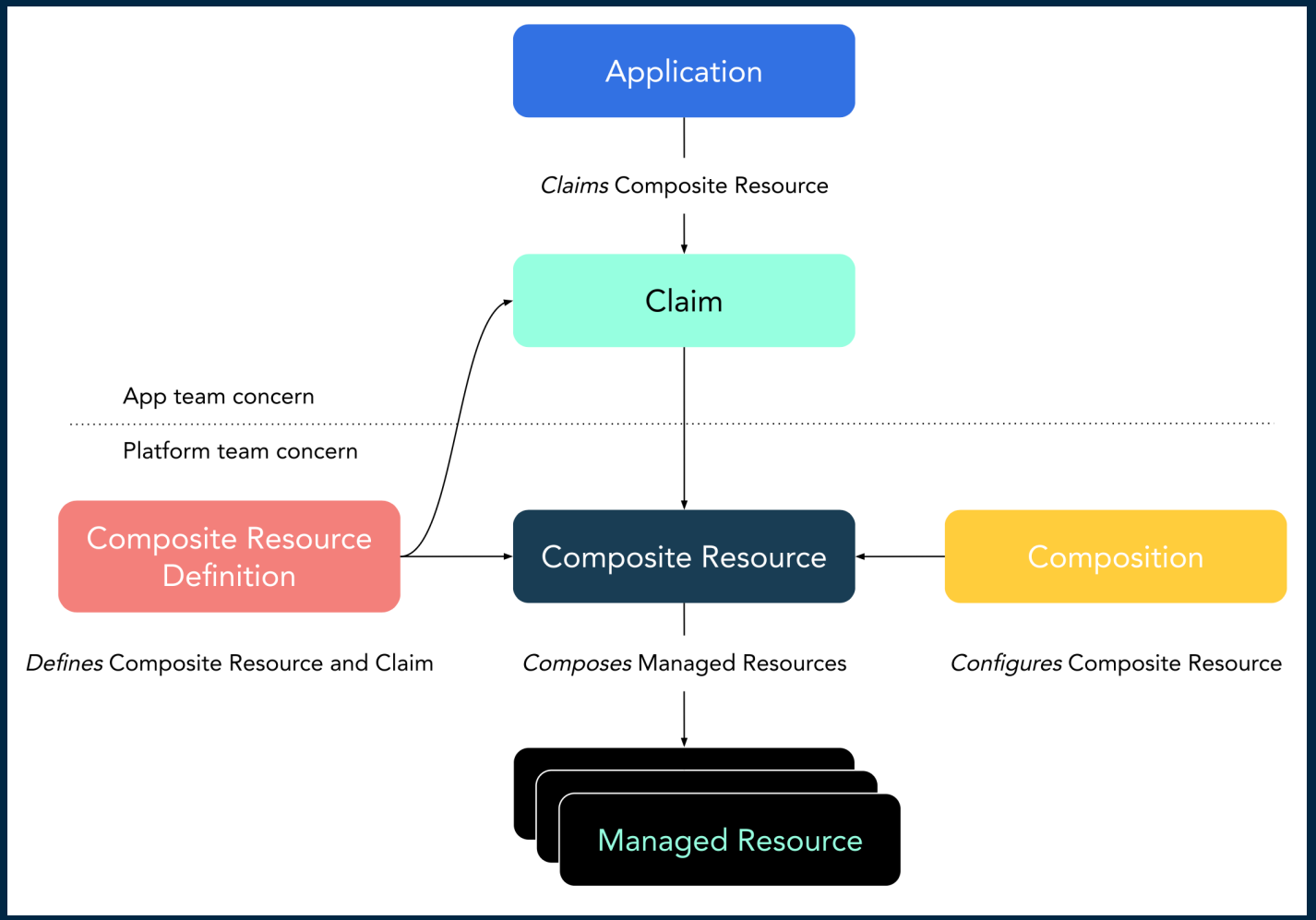
> cat user_wish_list.txt

- Selvbetjening
- Abstraksjoner som gir mening
  - Altså at man egentlig ikke trenger å vite så mye om Azure
- Et grensesnitt vi liker
- Likt grensesnitt på tvers av flere skyer
- Overvåking og korrigering av avvik

Crossplane.io

- Åpen kildekode
- Rammeverk for plattform-utvikling
- Bygd på toppen av Kubernetes
- Sky-agnostisk, støtte for AWS, Azure, GCP ++
- Korrigerer avvik automatisk
- Lar oss bygge abstraksjon på toppen av underliggende infrastruktur





"Diagram © Crossplane, used under CC-BY 4.0"

HÆ?! Skjønner ingenting. Vis oss noen eksempler!



Byggesteinene

Crossplane	Utvider Kubernetes med nye API-er for å koordinere overvåking og korrigering av avvik
Providers	Støtte for `Managed Resources` hos ulike sky-leverandører
Managed Resources	Gjør en ressurs i f.eks. Azure tilgjengelig som en ressurs i Kubernetes
Composite Resources	Brukerens grensesnitt for å opprette ressurser de trenger
Compositions	Definisjonen av hvordan en `Composite Resource` blir implementert
Functions	Gjør `Compositions` fleksible og gjenbrukbare

Nice! Vi vil se mer!



Det viktigste vi har lært

1. Bratt læringskurve (for plattform-teamet)
2. Vanskeligere å finne gode folk med relevant erfaring
3. Det lønner seg å bruke tid på gode API-design tidlig
4. Gode teknologer er ikke nødvendig gode produkt-folk
5. Involverte brukere == fornøyde brukere

Q & A

> git clone demo-material

https://github.com/sjovang/azure-crossplane-demo

Thank You

MVP-Dagen



Trond Jakob Sjøvang, Azure MVP

Manager @ Sopra Steria